Web Engineering & Development (SWE 363)

MongoDB & Mongoose

In today's lecture:

- MongoDB: documents, collections, and the MongoDB Shell
- CRUD operations in the shell
- Mongoose: schemas, models, and CRUD in Node.js

Reference:

- Zybook: 7.1, 7.2

Activity: Think-Pair-Share (2 min)

Recall 6.1: a new product with an extra color field is added to both databases.

```
INSERT INTO Product VALUES (999, 315.99, 'Smart watch', 'blue');
```

```
db.products.insertOne({ _id: 999, price: 315.99, title: "Smart watch", color: "blue" })
```

1. A relational database stores data in _____; a document database stores documents in _____ format.
2. Why does the SQL statement fail while the MongoDB command works?
3. What could a table, a row, and a column be called in MongoDB?

7.2 MongoDB

What is MongoDB?

> **MongoDB** is the most popular **NoSQL** database used by Node.js developers

- **Document** → a single data object made of **field/value pairs** (like JSON)
- **Collection** → a group of related documents
- Documents are stored internally as **BSON** (Binary JSON)
  - JSON + extra type information: string, integer, double, date, boolean, null, ...
  - A BSON document may not exceed **16 MB**

MongoDB Structure

A document

```
{
  name: "Sue",          // string
  gpa: 3.1,             // double
  interests: ["biking", "reading"],
  address: {            // nested
    city: "Dallas",
    state: "TX"
  }
}
```

A collection

```
// students collection
{ name: "Sue", gpa: 3.1, ... }
{ name: "Larry", gpa: 2.5, ... }
{ name: "Anne", gpa: 4.0, ... }
```

```
mydb database
  → students collection
    → documents
```

MongoDB Shell (mongosh)

- **MongoDB Shell** → a command-line interface; the `mongosh` command starts it
- The MongoDB installer also includes **MongoDB Compass**, a GUI for MongoDB

```
$ mongosh
Connecting to: mongodb://127.0.0.1:27017/...
test> use mydb                  // creates or selects the "mydb" database
switched to db mydb
mydb> db.students.insertOne({ _id: 123, name: "Sue", gpa: 3.1 })
{ acknowledged: true, insertedId: 123 }
mydb> db.students.find()
[ { "_id" : 123, "name" : "Sue", "gpa" : 3.1 } ]
mydb> show dbs                  // lists all databases
mydb> show collections          // lists all collections in the current database
mydb> help                      // summary of all shell commands
mydb> exit                      // quits the MongoDB Shell
```

Activity: Classify, then name the command

```
test> use dealer
dealer> db.autos.insertOne({ _id: 200, make: "Ford", model: "Mustang" })
```

Database, collection, document, field, or value?

1. `dealer`
2. `autos`
3. `{ _id: 200, make: "Ford", model: "Mustang" }`
4. `make`
5. `"Ford"`

Inserting Documents

- `insertOne()` → **one** document; `insertMany()` → **multiple** documents
- `_id` → the **primary key**: uniquely identifies each document; always the first field
- No `_id` given → MongoDB assigns an **ObjectId** (12-byte BSON type, shown in hex)

```
mydb> db.students.insertOne({ name: "Sue", gpa: 3.1 })
{ acknowledged: true, insertedId: ObjectId("62794229fc4ebd4933877a9d") }
mydb> students = [ { name: "Maria", gpa: 4.0 }, { name: "Xiu", gpa: 3.8 },
... { name: "Braden", gpa: 2.5 } ]
mydb> db.students.insertMany(students)
{ acknowledged: true,
  insertedIds: { '0': ObjectId("627942f6fc4ebd4933877a9e"),
                 '1': ObjectId("627942f6fc4ebd4933877a9f"),
                 '2': ObjectId("627942f6fc4ebd4933877aa0") } }
```

Finding Documents & Query Operators

- `find()` → returns **all** documents, or those that match an optional query
- `findOne()` → returns only the **first** matching document (`null` if none)

```
db.students.find({ name: 'Sue' })                        // field: value → equal to the value
db.students.find({ name: { $ne: "Sue" } })               // $eq / $ne    → = or ≠ the value
db.students.find({ gpa: { $gt: 3.5 } })                  // $gt / $gte   → > or ≥ the value
db.students.findOne({ gpa: { $lte: 3.0 } })              // $lt / $lte   → < or ≤ the value
db.students.find({ name: { $in: ["Sue", "Susie"] } }) // $in / $nin   → in or not in an array

// $and / $or → matches both / either clause: gpa >= 3.9 or gpa <= 3.0
db.students.find({ $or: [{ gpa: { $gte: 3.9 } }, { gpa: { $lte: 3.0 } }] })
```

Updating Documents

- `updateOne()` / `updateMany()` → modify **one** / **multiple** documents
- Parameters: a **query** (`{}` matches **all**) and an **update operator**

```
mydb> db.students.updateOne({ name: 'Sue' }, { $set: { gpa: 3.3 } })
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }
```

```
{ $set: { gpa: 4.0 } }                    // $set → sets a field's value
{ $inc: { gpa: 0.1 } }                    // $inc → increments a field's value by the amount
{ $unset: { gpa: "", birthDate: "" } }    // $unset → removes fields
{ $rename: { name: 'firstName' } }        // $rename → renames a field
{ $currentDate: { birthDate: true } }     // $currentDate → sets the current date/time
```

Deleting Documents

- `deleteOne()` / `deleteMany()` → delete **one** / **multiple** matching documents
- Returns `deletedCount` → how many documents were deleted

```
mydb> db.students.deleteOne({ gpa: { $lt: 3.5 } })
{ acknowledged: true, deletedCount: 1 }

mydb> db.students.deleteMany({ gpa: { $gt: 3.5 } })
{ acknowledged: true, deletedCount: 2 }

mydb> db.students.find()
[ { _id: ObjectId("627942f6fc4ebd4933877aa0"), name: 'Braden', gpa: 2.5 } ]
```

Activity: Predict: which documents are returned (1–3)? What changes (4–5)?

_id	make	year	price
100	Ford	2014	13500
200	Honda	2013	16900
300	Dodge	2012	10800
400	Toyota	2013	13400

Key: `$gt` >, `$gte` ≥, `$lt` <, `$ne` ≠, `$or` either condition, `$set` sets a field

```
db.autos.find({ year: { $gte: 2013 } })                                          // 1.
db.autos.findOne({ year: { $gte: 2013 } })                                       // 2.
db.autos.find({ $or: [ { make: "Honda" }, { year: { $ne: 2013 } } ] })           // 3.
db.autos.updateOne({ price: { $gt: 13000 } }, { $set: { sold: true } })          // 4.
db.autos.deleteMany({ price: { $lt: 13500 } })                                   // 5.
```

7.1 Mongoose

What is Mongoose?

> The **mongoose** module provides **object data mapping (ODM)** and structured schemas to MongoDB collections

- **ODM** → converts data from a database into a **JavaScript object**
- The `mongodb` module also works, with many of the same functions as the shell
- Many developers prefer Mongoose → it **simplifies** some MongoDB operations

```
npm install mongoose
```

```
const mongoose = require("mongoose");
```

Mongoose Schemas

> A **schema** defines the structure of documents within a MongoDB collection

```
const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  gpa: { type: Number, min: 0, max: 4 },
  birthDate: { type: Date, default: Date.now },
  interests: [ String ]
});
```

Data types: `String`, `Number`, `Date`, `Buffer` (binary), `Boolean`, `Mixed` (any), `ObjectId`, `Array` (e.g., `[Number]`)

Mongoose Models

> A **model** is a constructor compiled from a schema

```
// Create a "Student" model from the studentSchema schema
const Student = mongoose.model("Student", studentSchema);

// Create a student document from the Student model
const stu = new Student({
  name: "Sue Black",
  gpa: 3.1,
  birthDate: new Date(2006, 11, 2),
  interests: ["biking", "reading"]
});
```

- Model name `"Student"` → collection **"students"** (the plural)
- Good practice: capitalize model names and variables (`Card` , not `card`)

Connecting & Saving Documents

```
const express = require("express");
const mongoose = require("mongoose");
mongoose.connect("mongodb://localhost/mydb");
// ... studentSchema and Student model ...
const app = express();
app.get("/create", async function(req, res) {
  const stu = new Student({ name: "Sue Black", gpa: 3.1 });
  await stu.save();
  res.send(`Saved ${stu.name}.`);
});
app.listen(8080);
```

- `connect()` → connects **before** any operations; creates `mydb` if needed
- `await stu.save()` → saves the document and assigns its `_id`
- `Student.create()` → also saves one document or an array of documents

Activity: What happens if… (each change on its own)

```
mongoose.connect("mongodb://localhost/mydb");

const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  birthDate: { type: Date, default: Date.now }
});
// Model "Student" → collection "students"
const Student = mongoose.model("Student",
  studentSchema);

app.get("/create", async function(req, res) {
  const stu = new Student({ name: "Sue Black" });
  await stu.save();
  res.send(`Saved ${stu.name}.`);
});
```

1. A student is saved with no `birthDate`?
2. A student is saved with no `name`?
3. `mongoose.model("Dog", dogSchema)`: which collection?
4. The URL ends in `/students` instead of `/mydb`?
5. `await` is removed before `stu.save()`?

Finding Documents with Mongoose

Method	Finds
`Model.find()`	All documents that match the conditions
`Model.findOne()`	The first document that matches
`Model.findById()`	The document with the given id

```
// Find all students with gpa >= 3
const students = await Student.find({ gpa: { $gte: 3 } });

// Find the student with this ObjectId
const stu = await Student.findById("66901e8c09a55e28d1acc296");
if (stu === null) {
  console.log("No student found");
}
```

Chaining Query Methods

All find methods return a **Query** object → chain methods for a readable query

```
async function findStudents() {
  const students = await Student.find({ gpa: { $gte: 3 } })
    .where("interests").in(["fishing", "biking"])
    .sort({ "birthDate": "desc" })
    .limit(2)
    .exec();
  students.forEach(function(stu) {
    console.log(stu.name);
  });
}
```

- `where()` → filter on a path; `sort()` → set the sort order
- `limit()` → limit the number of documents; `exec()` → called **last** to run the query

Activity: Predict the output of findStudents()

```
Student.find({ gpa: { $gte: 3 } }).where("interests").in(["fishing", "biking"])
  .sort({ "birthDate": "desc" }).limit(2).exec();   // then prints each stu.name
```

name	gpa	birthDate	interests
Sue	3.1	2006-11-2	biking, reading
Mara	2.6	2002-5-6	dancing, coding
Dana	3.3	2008-1-15	gardening, biking
Li	4.0	2001-2-6	rock climbing
Greg	3.2	1998-6-30	fishing, tennis

1. What is printed?
2. What if `.limit(2)` becomes `.limit(3)`?
3. What if `"desc"` becomes `"asc"`?
4. What if the `.where(...)` line is removed?

Update & Delete Methods

Update

```
// Change Sue's birthDate to 2008-12-02
const result = await Student.updateOne(
  { name: "Sue Black" },
  { birthDate: new Date(2008, 11, 2) });

// Add 0.5 to all GPAs <= 3.5
const result = await Student.updateMany(
  { gpa: { $lte: 3.5 } },
  { $inc: { gpa: 0.5 } });

// save() also updates a found document
stu.gpa += 0.1;
await stu.save();
```

Delete

```
// Delete student with given id
const result = await Student.deleteOne(
  { _id: "668fe18cda9cd0d66eee9a76" });

// Delete all students with GPA < 3
const result = await Student.deleteMany(
  { gpa: { $lt: 3 } });
console.log("Docs deleted: " +
  result.deletedCount);
```

Activity: Exit ticket - find the 5 bugs

```
const mongoose = require("mongoose");
mongoose.connect("http://localhost/mydb");

const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  gpa: { type: Number, min: 0, max: 4 }
});
const Student = mongoose.model("Student", studentSchema);

async function addStudent() {
  const stu = new Student({ gpa: 3.5 });
  stu.save();
  const honors = await Student.find({ gpa: { gte: 3.5 } });
  await Student.deleteOne(123);
}
```

Key Takeaways

- MongoDB stores **documents** (field/value pairs, saved as BSON) in **collections**
- **mongosh:** `use`, `show dbs`, `show collections`, `db.collection.method()`
- **CRUD:** `insertOne/Many`, `find/findOne`, `updateOne/Many`, `deleteOne/Many`
- **Query operators** (`$gt`, `$in`, `$or`) filter; **update operators** (`$set`, `$inc`) modify
- **Mongoose:** schema → model → document → `save()` or `Model.create()`
- Chain Mongoose **Query** methods: `where()`, `sort()`, `limit()`, `exec()`

30m

Demo

7.1 MongoDB

MongoDB & Mongoose hands-on:

- MongoDB and mongosh setup
- CRUD operations in mongosh
- Mongoose connection, schema, and CRUD

Next Class

- RESTful web APIs (7.3)
- Using RESTful web APIs with Fetch (7.5)